Elyra AI · Developer task

Order Event Processor

Kotlin, PostgreSQL, Redis

The data

Attached is a file `events.jsonl`. Every line is an event for an order, like this:

```
{"eventId":"e7f3c1","orderId":"ord-1042","sequence":3,"type":"OrderPaid",
 "occurredAt":"2026-10-20T14:02:11Z","payload":{"amountCents":12990,"currency":"EUR"}}
```

An order goes through `OrderCreated` (with `items`: `sku`, `qty`, `unitPriceCents`), then any number of `ItemAdded` (same fields), then `OrderPaid` (`amountCents` must equal the sum of all items), then `OrderShipped` (`trackingId`). It can be cancelled with `OrderCancelled` any time before shipping. `sequence` counts from 1 per order and tells you the real order of events. Ignore `occurredAt`, the clocks that produced it were not in sync.

The file is a bit of a mess. Some events are in there twice, some come before the event they depend on, a few lines are simply broken, and one order has two different events claiming the same sequence number.

Your task

Write a Kotlin program that processes the file. We expect the following:

1. The program reads the file and ends up with the correct current state of every order (status, total, last sequence) in a PostgreSQL table, and the status in Redis under `order:{orderId}:status`.
2. It gets the right result even if you kill it with `kill -9` in the middle and start it again.
3. Events that can't be applied yet because an earlier one is missing wait; they are not thrown away.
4. Events that can't be applied at all (bad payload, wrong amount, not allowed in the order's current state) are rejected with a reason and kept somewhere we can look at later. Their sequence number still counts, so the order continues with the next one.
5. Redis never shows a status that isn't committed in PostgreSQL.
6. At the end, the program prints how many events were applied, skipped as duplicates, still waiting, and rejected.
7. At least one test actually goes red if you take out the ordering logic.

Submission

Send us the git repo with a short note on how to run it. Where this description doesn't say what to do, pick a sensible rule, apply it consistently, and state it in that note.

Docker Compose for PostgreSQL and Redis is fine. Gradle, JVM 17 or newer, no framework needed.